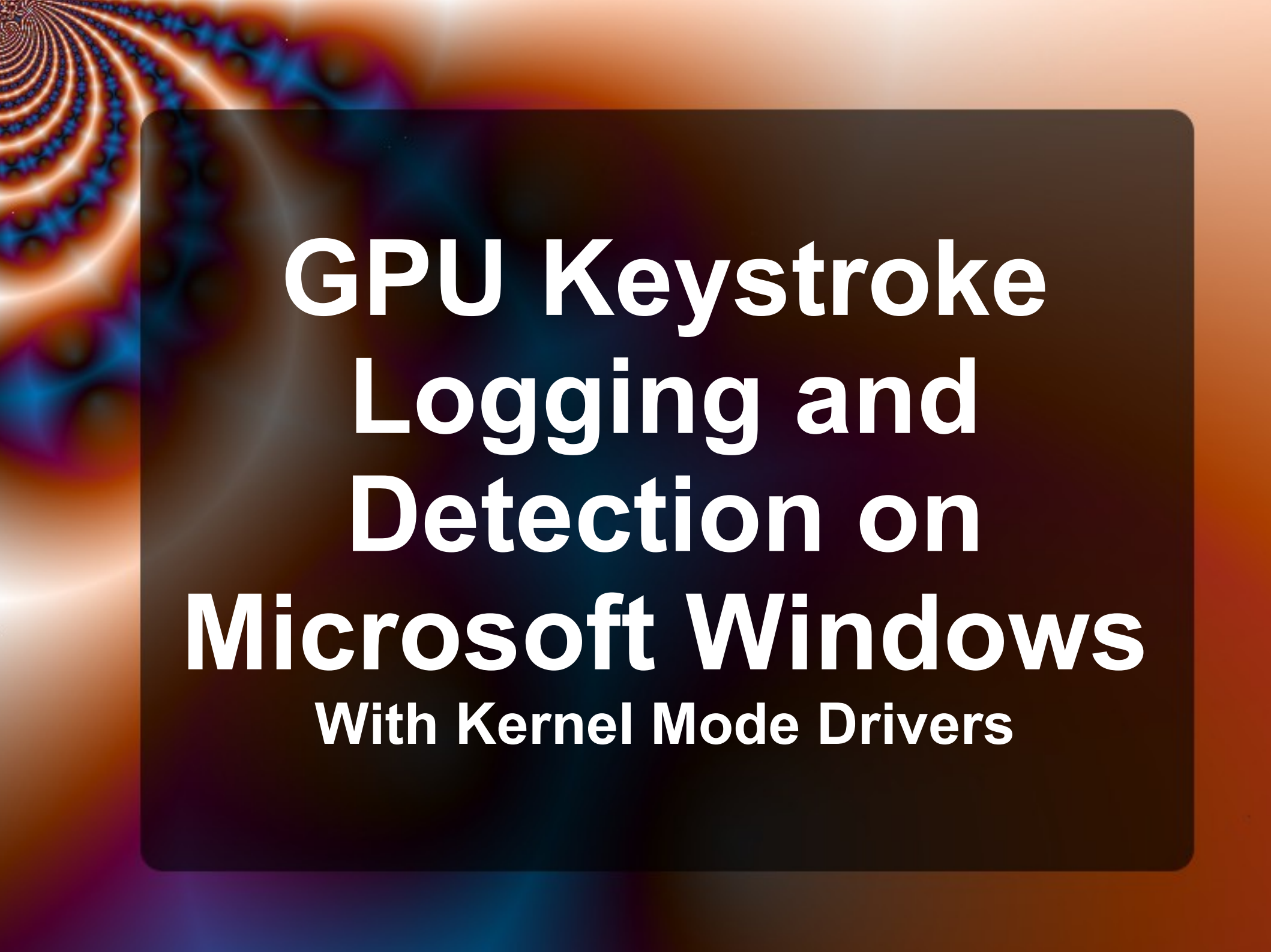
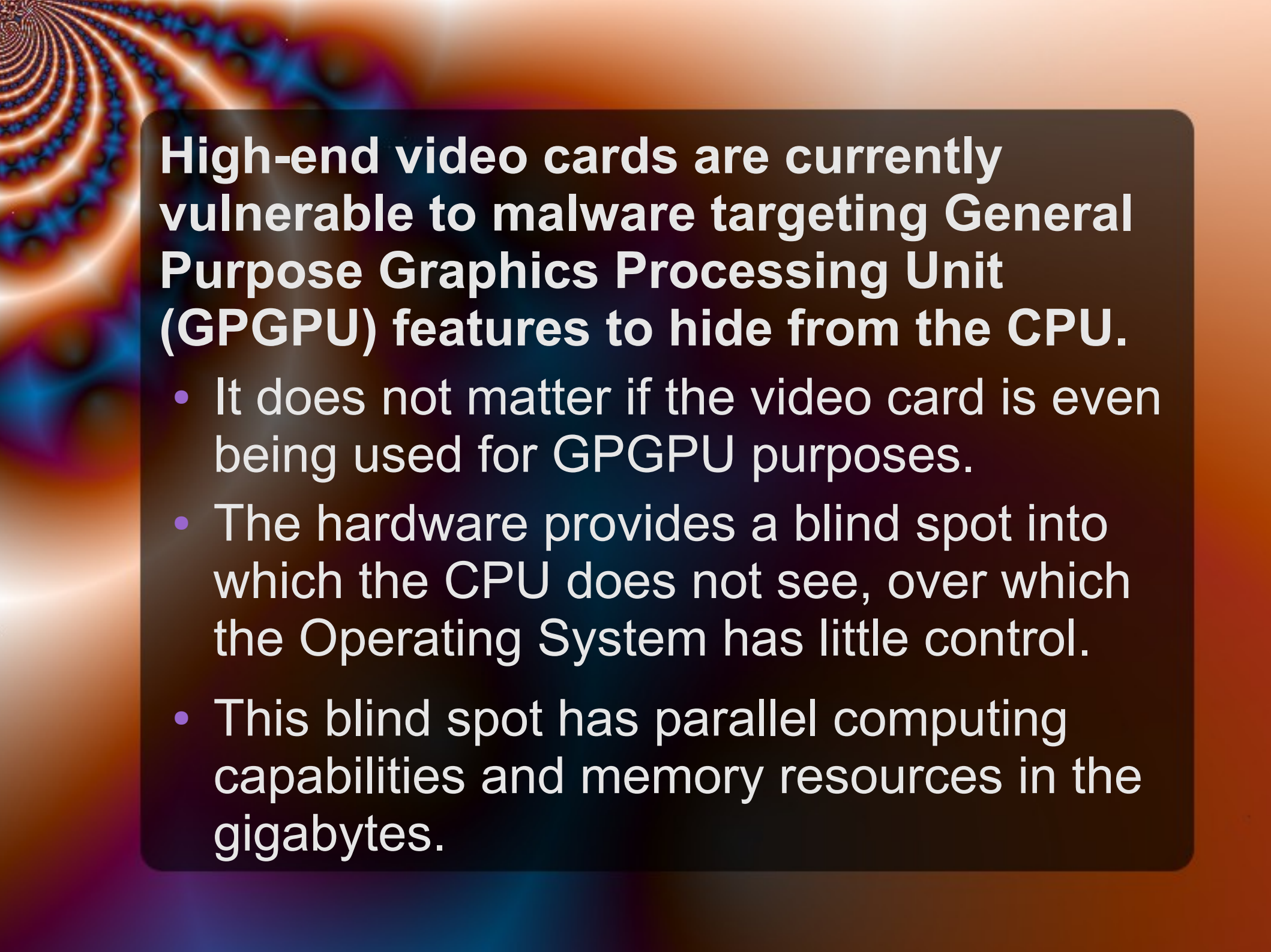


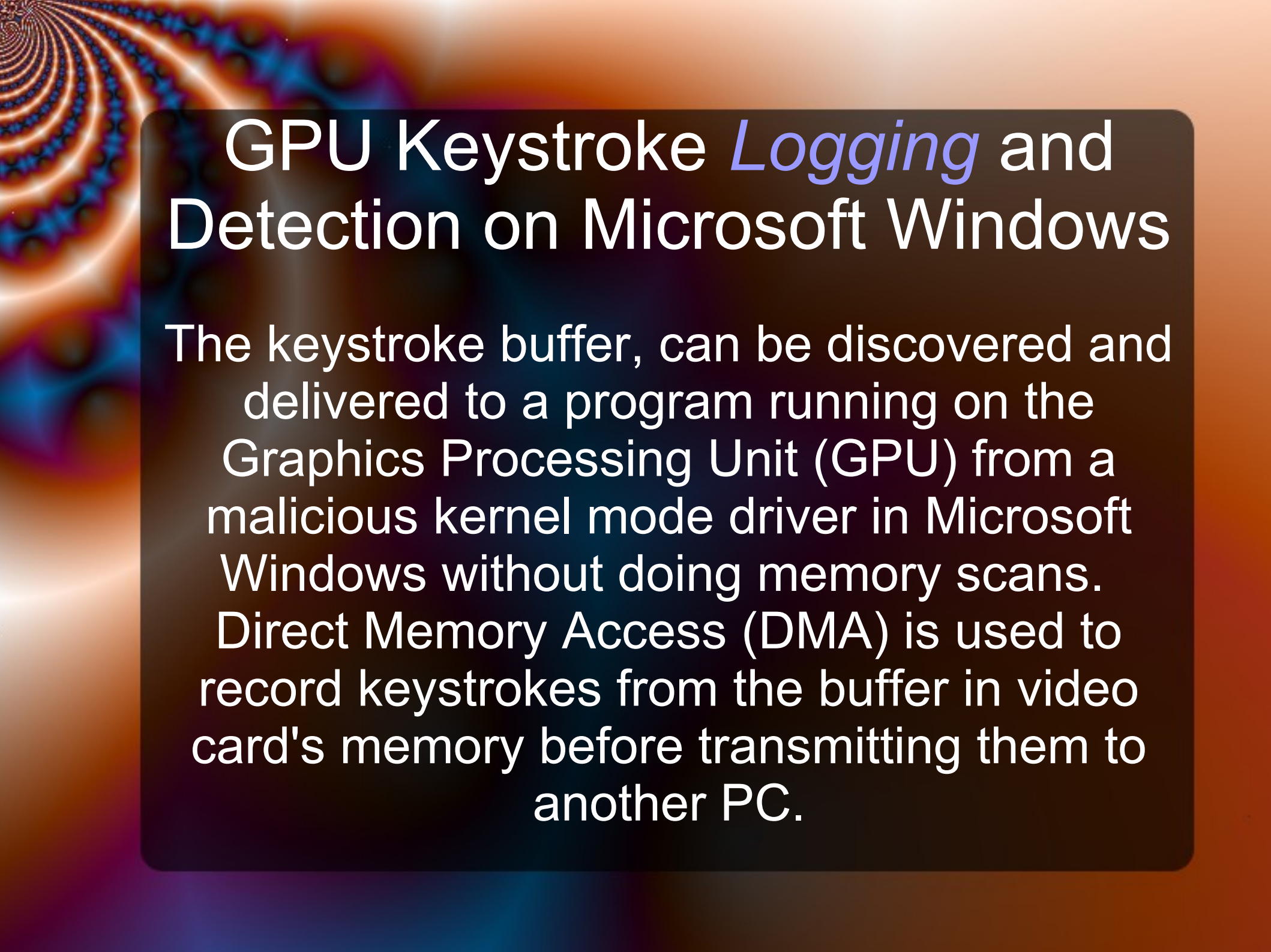


GPU Keystroke Logging and Detection on Microsoft Windows With Kernel Mode Drivers



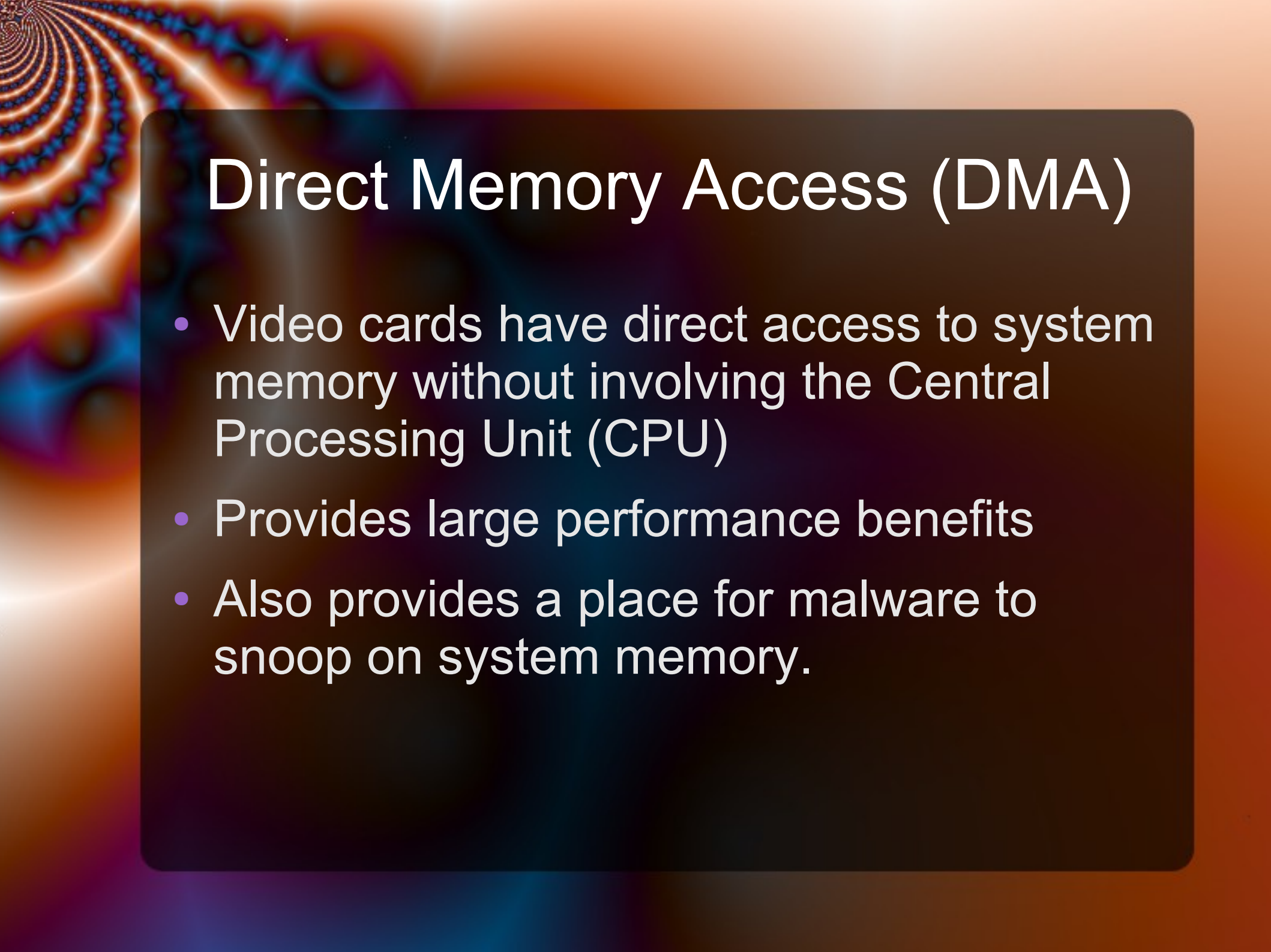
High-end video cards are currently vulnerable to malware targeting General Purpose Graphics Processing Unit (GPGPU) features to hide from the CPU.

- It does not matter if the video card is even being used for GPGPU purposes.
- The hardware provides a blind spot into which the CPU does not see, over which the Operating System has little control.
- This blind spot has parallel computing capabilities and memory resources in the gigabytes.



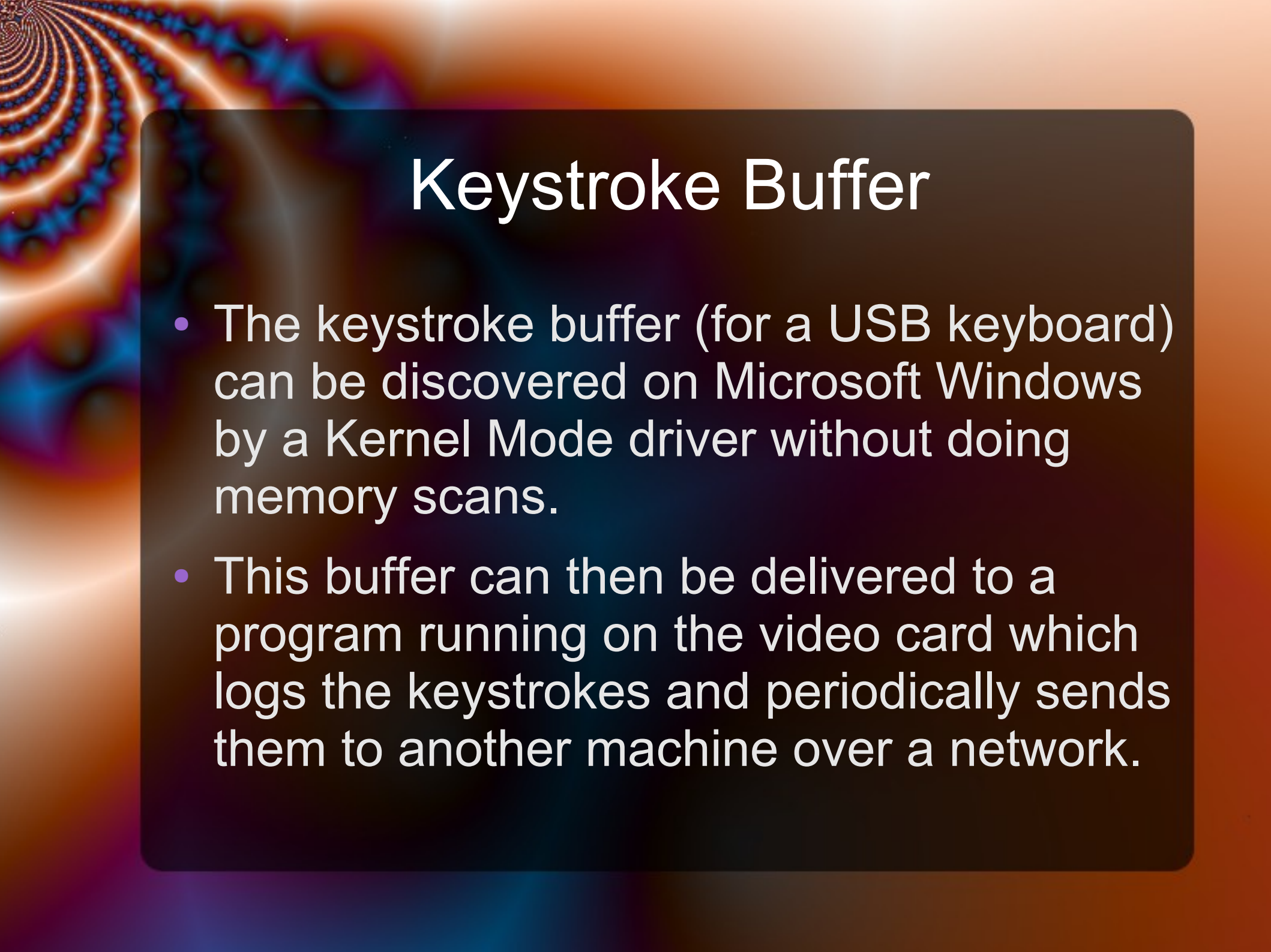
GPU Keystroke *Logging* and Detection on Microsoft Windows

The keystroke buffer, can be discovered and delivered to a program running on the Graphics Processing Unit (GPU) from a malicious kernel mode driver in Microsoft Windows without doing memory scans. Direct Memory Access (DMA) is used to record keystrokes from the buffer in video card's memory before transmitting them to another PC.



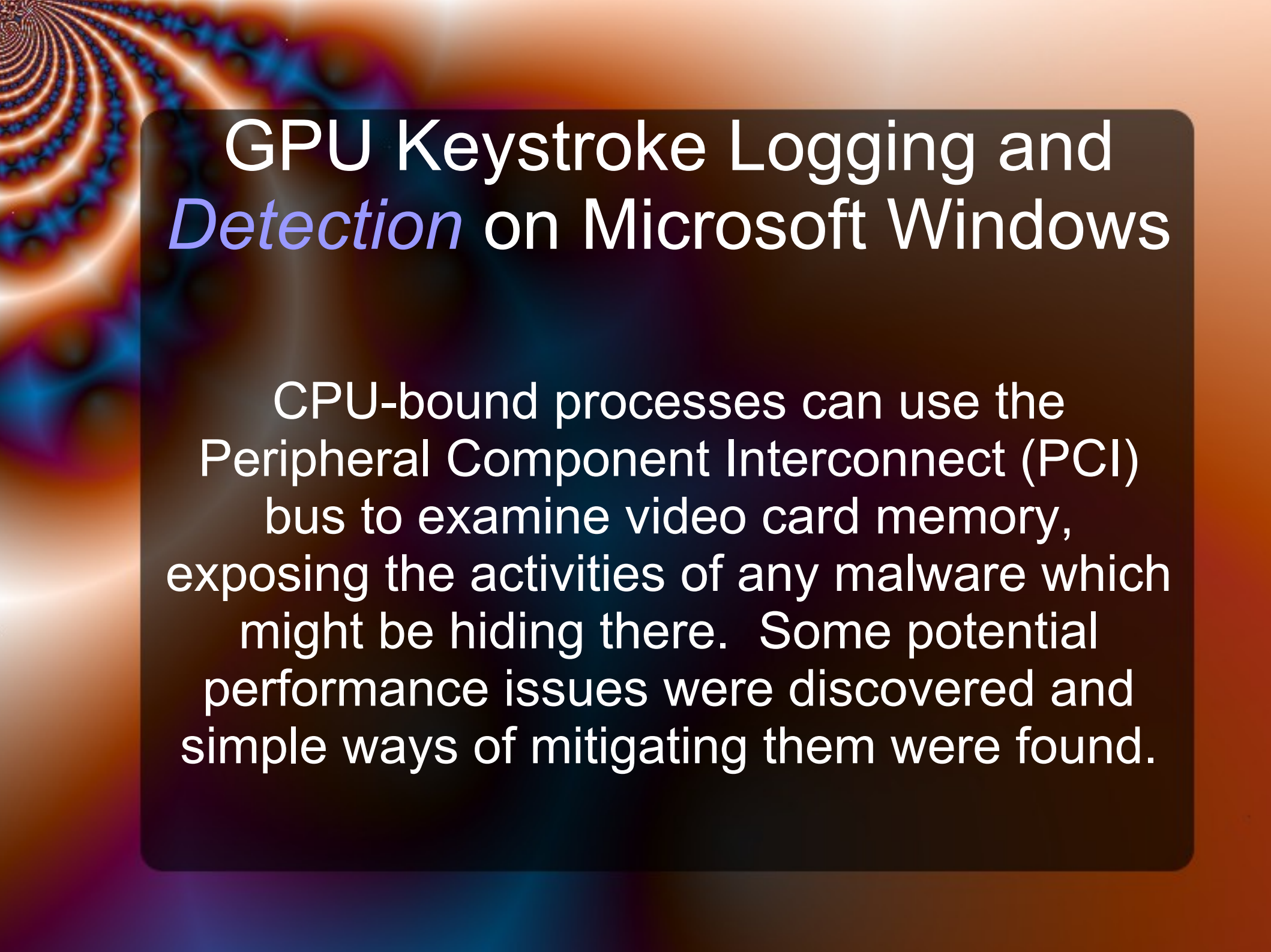
Direct Memory Access (DMA)

- Video cards have direct access to system memory without involving the Central Processing Unit (CPU)
- Provides large performance benefits
- Also provides a place for malware to snoop on system memory.



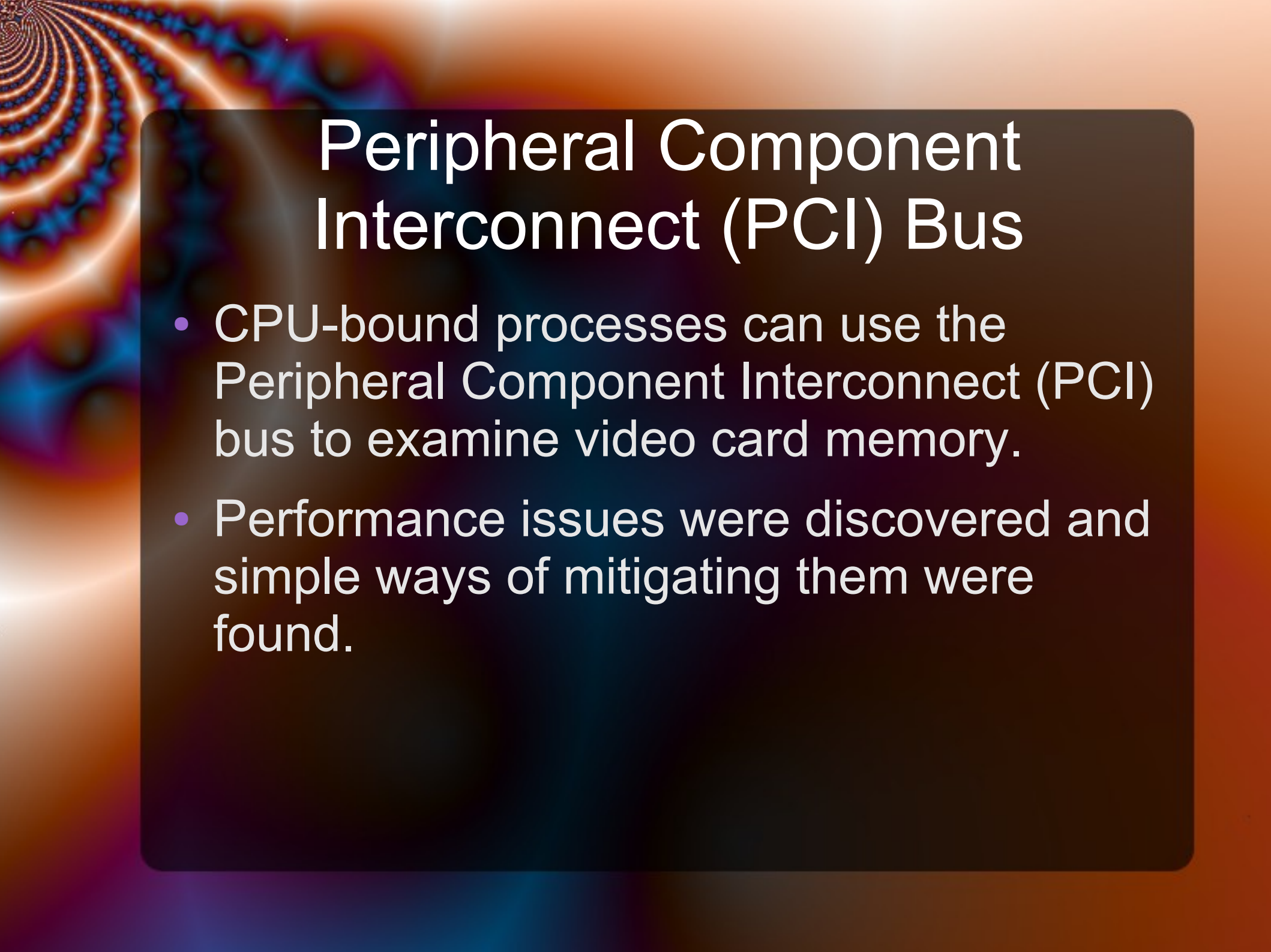
Keystroke Buffer

- The keystroke buffer (for a USB keyboard) can be discovered on Microsoft Windows by a Kernel Mode driver without doing memory scans.
- This buffer can then be delivered to a program running on the video card which logs the keystrokes and periodically sends them to another machine over a network.



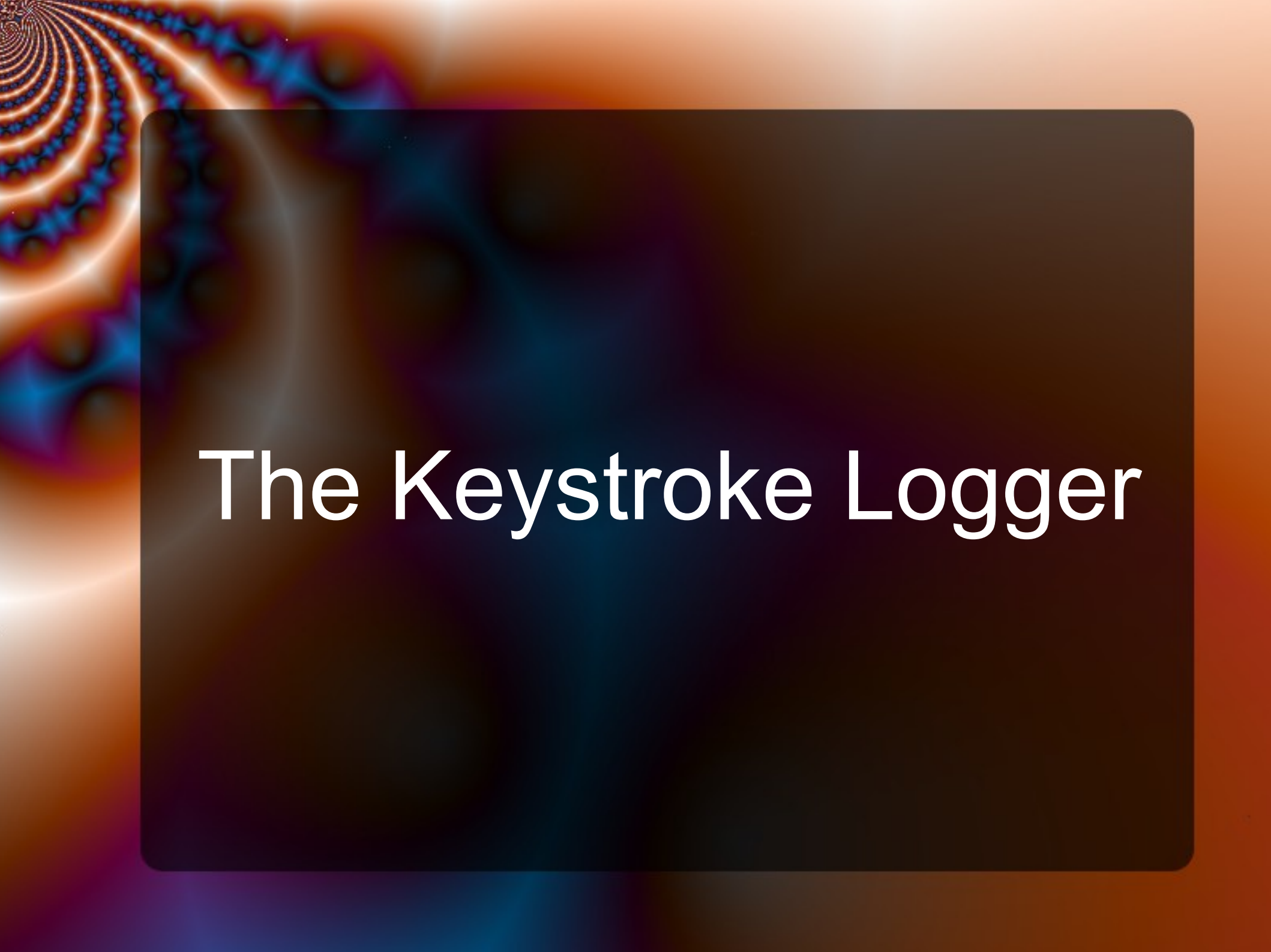
GPU Keystroke Logging and *Detection* on Microsoft Windows

CPU-bound processes can use the Peripheral Component Interconnect (PCI) bus to examine video card memory, exposing the activities of any malware which might be hiding there. Some potential performance issues were discovered and simple ways of mitigating them were found.

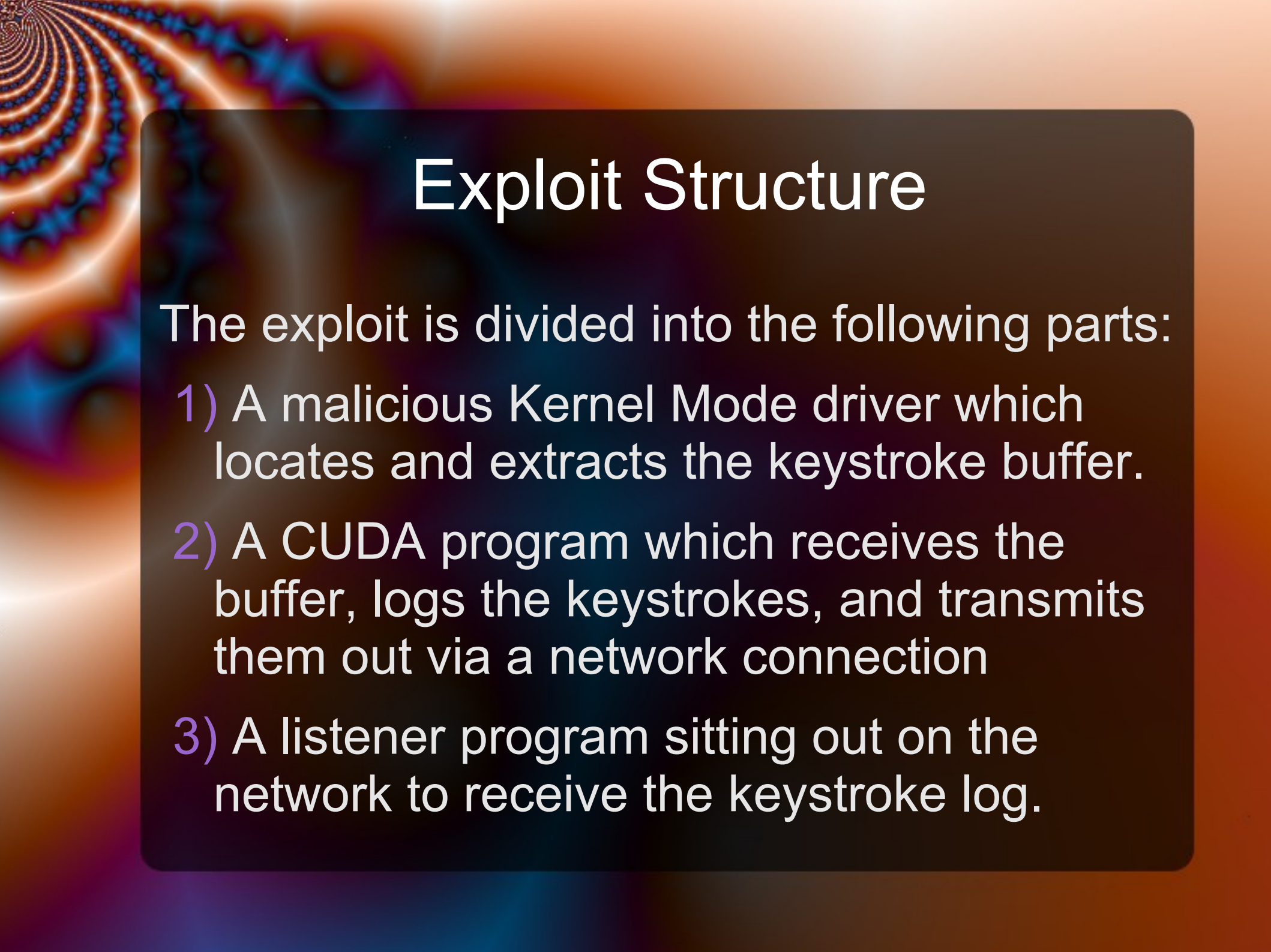


Peripheral Component Interconnect (PCI) Bus

- CPU-bound processes can use the Peripheral Component Interconnect (PCI) bus to examine video card memory.
- Performance issues were discovered and simple ways of mitigating them were found.



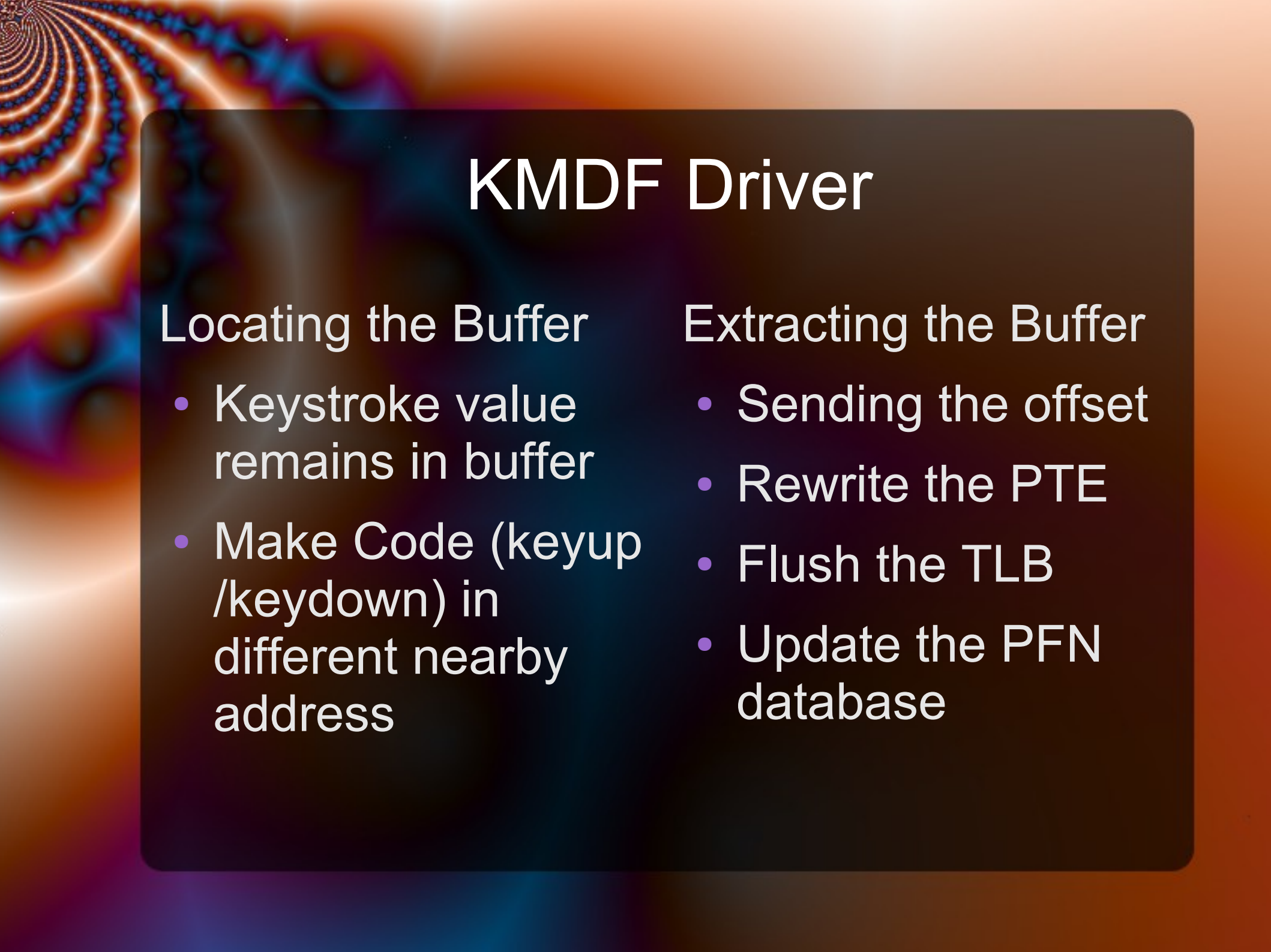
The Keystroke Logger



Exploit Structure

The exploit is divided into the following parts:

- 1) A malicious Kernel Mode driver which locates and extracts the keystroke buffer.
- 2) A CUDA program which receives the buffer, logs the keystrokes, and transmits them out via a network connection
- 3) A listener program sitting out on the network to receive the keystroke log.



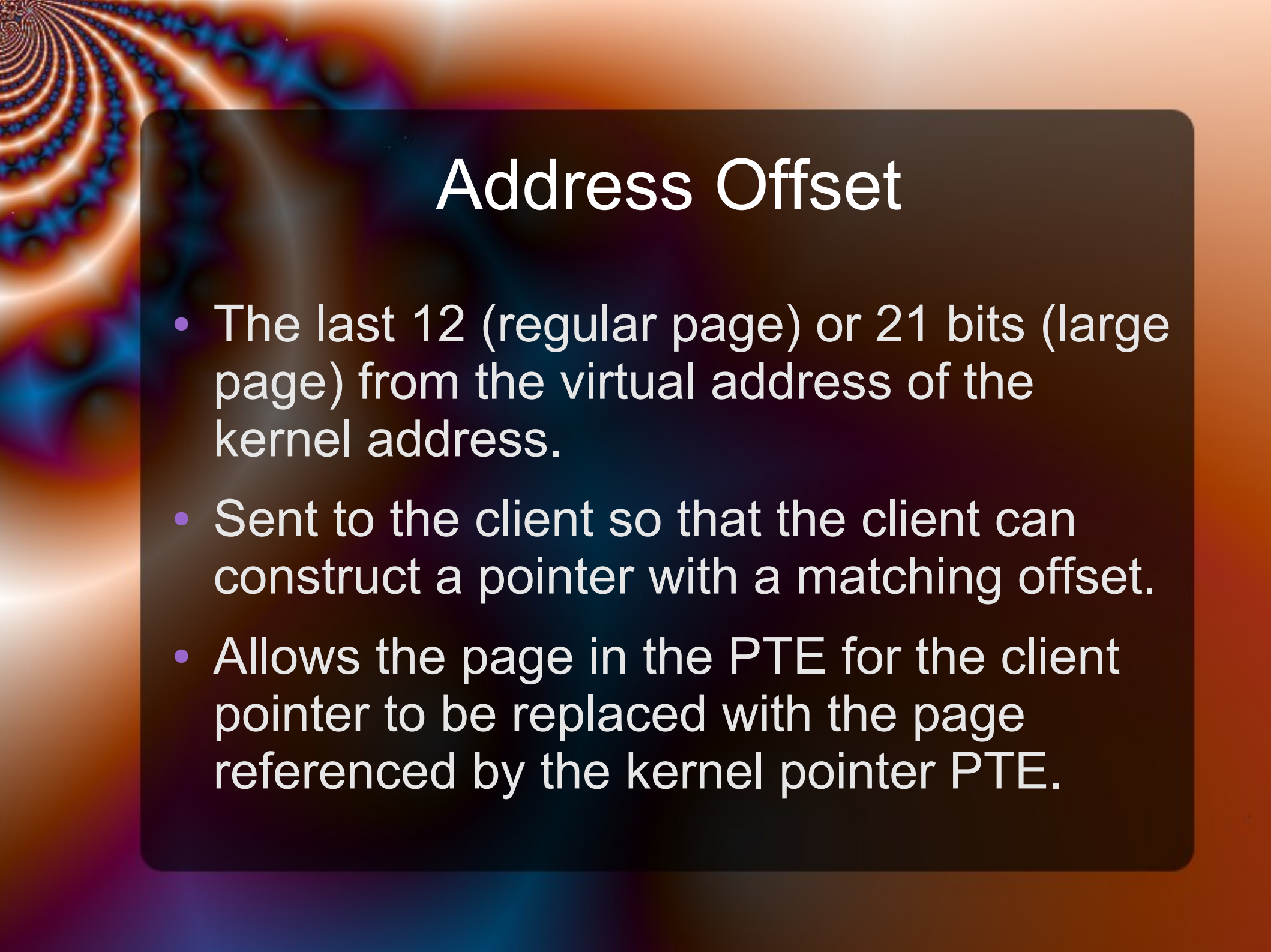
KMDF Driver

Locating the Buffer

- Keystroke value remains in buffer
- Make Code (keyup /keydown) in different nearby address

Extracting the Buffer

- Sending the offset
- Rewrite the PTE
- Flush the TLB
- Update the PFN database



Address Offset

- The last 12 (regular page) or 21 bits (large page) from the virtual address of the kernel address.
- Sent to the client so that the client can construct a pointer with a matching offset.
- Allows the page in the PTE for the client pointer to be replaced with the page referenced by the kernel pointer PTE.

Rewrite the PTE

32-Bit Addressing



- The 20 leftmost bits are split into a 10 bit Page Directory Entry (PDE) and a 10 bit Page Table Entry (PTE).
- PDE located at the Page Directory Base (PDB) address (0xC0300000) added to the product of the Entry # with the PDE size (4 bytes.)

Rewrite the PTE

Physical Address Extension



- 20 most significant bits are split into 3 parts:
 - First 2 bits are the Page Directory Pointer (PDP)
 - The next 9 are the Page Directory Entry
 - The last 9 are the Page Table Entry
- All entries are 8 bytes
- PDB address is 0xC0600000

Rewrite the PTE

Physical Address Extension



- The 4 PDEs are arranged in back to back virtual addresses, which means that one can use the first 11 bits as if they were the PDE address.

Rewrite the PTE

Large Pages



- Allocating a Large Page from user mode is problematic.
- Workaround: use regular address where the rightmost 21 bits of the address matched the rightmost 21 bits of the Kernel pointer address.
- PFN calculated based on the physical address of the buffer and the offset.

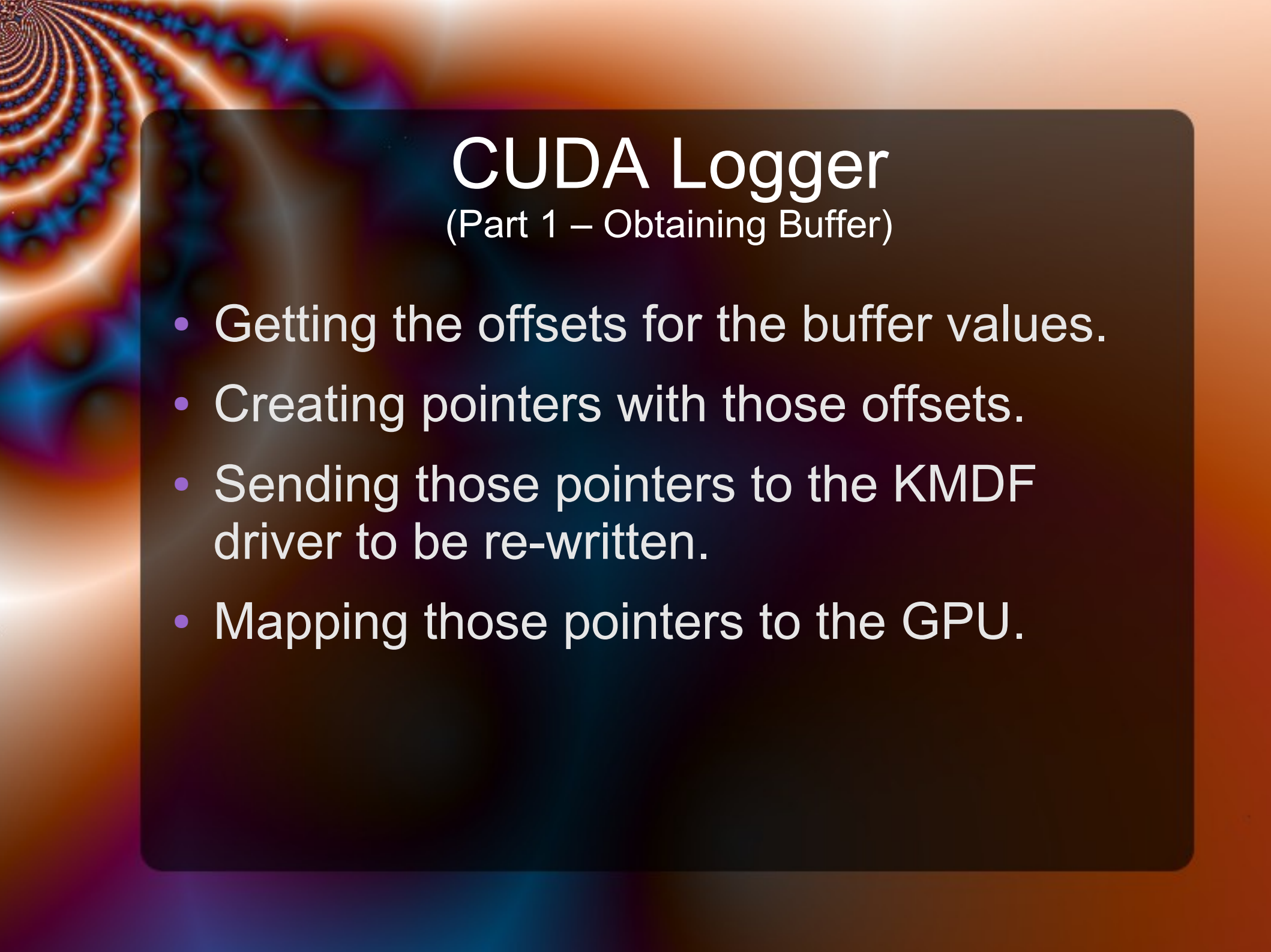
Flush the TLB

- Only part of the process which had to be done in assembly.

```
__asm __volatile  
{  
    cli  
    invlpg  clientPageTable;  
    sti  
}
```

Update the PFN Database

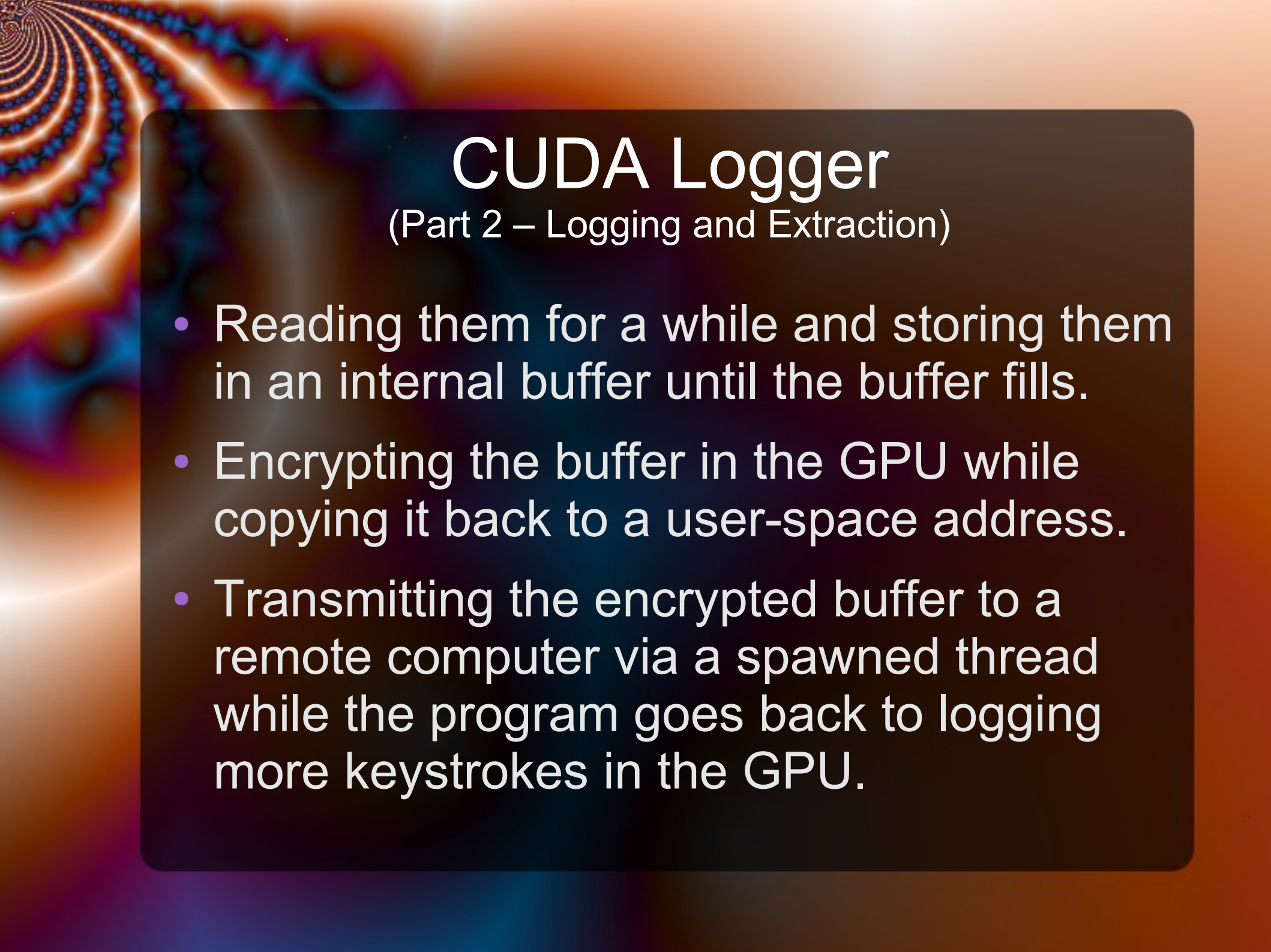
- PFN Database (PFND) keeps track pointer references for a memory address.
- Entry found by adding address times PFND Entry Size to the PFND base address.
 - 0x83A00000 was PFND base for my “victim” machine.
 - PFN entry size is 0x18 for 32 bit, and 0x1C for PAE.



CUDA Logger

(Part 1 – Obtaining Buffer)

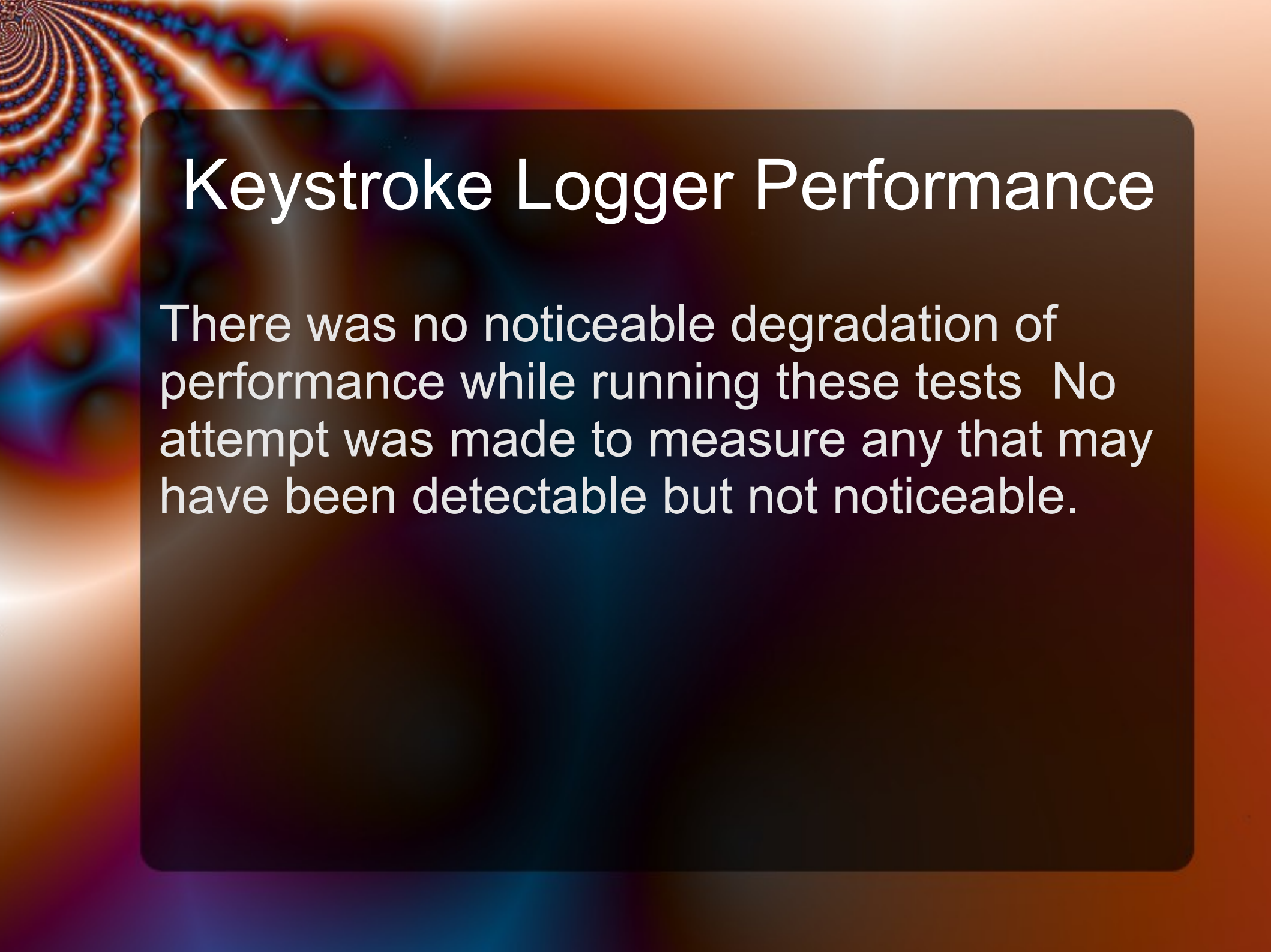
- Getting the offsets for the buffer values.
- Creating pointers with those offsets.
- Sending those pointers to the KMDF driver to be re-written.
- Mapping those pointers to the GPU.



CUDA Logger

(Part 2 – Logging and Extraction)

- Reading them for a while and storing them in an internal buffer until the buffer fills.
- Encrypting the buffer in the GPU while copying it back to a user-space address.
- Transmitting the encrypted buffer to a remote computer via a spawned thread while the program goes back to logging more keystrokes in the GPU.



Keystroke Logger Performance

There was no noticeable degradation of performance while running these tests. No attempt was made to measure any that may have been detectable but not noticeable.



PCI Video Memory Dump



PCI Video Memory Dump

- KMDF Driver to examine video memory
- Client to call driver and either examine or dump memory to disk.

PCI Video Memory Dump Driver

BAR 0

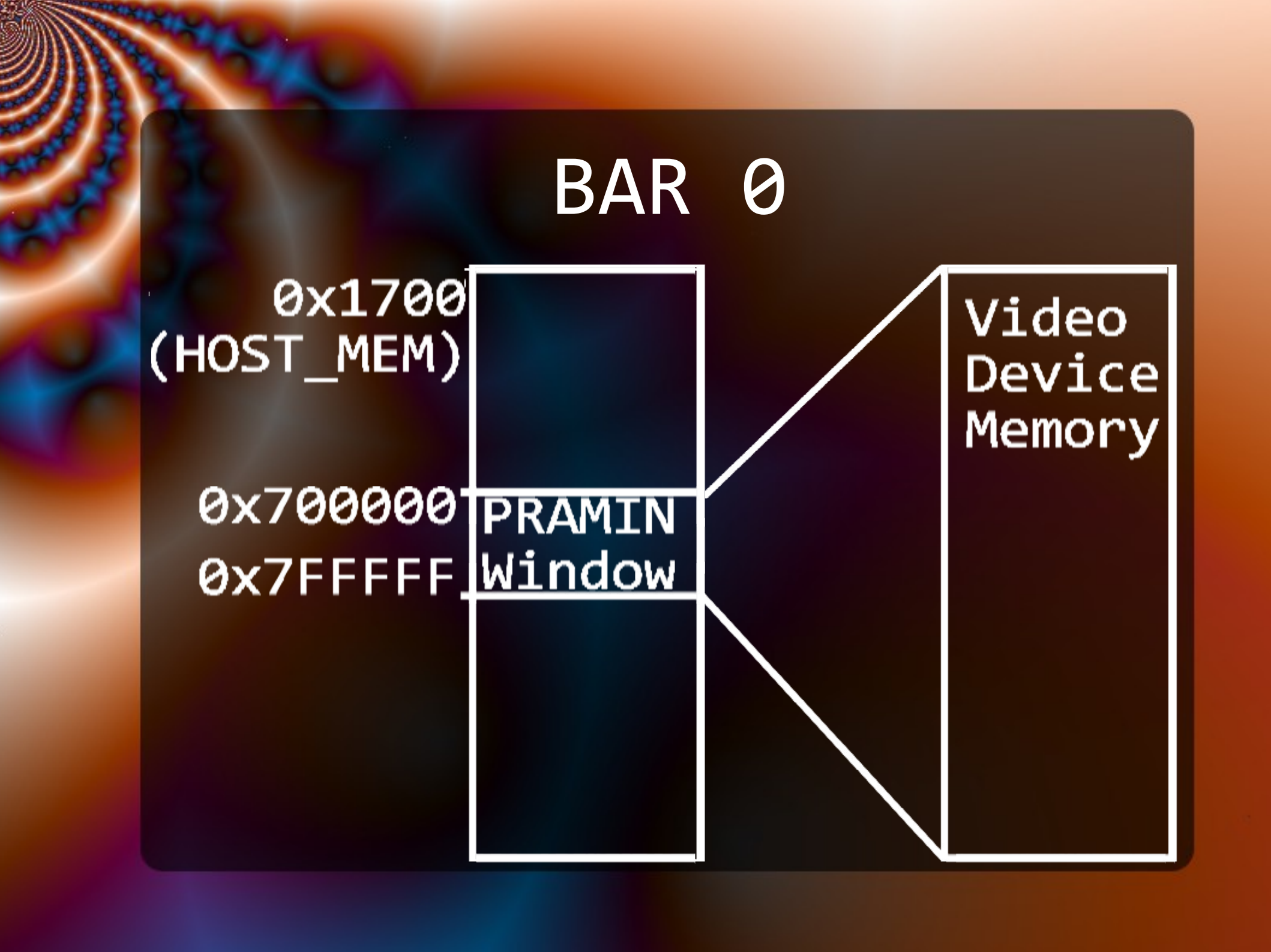
- Range of physical memory addresses which the BIOS maps to the video card.
- \Driver\pci driver object was obtained by call to ObReferenceObjectByName.
- Attached devices are iterated through until a video device (DeviceType = FILE_DEVICE_VIDEO or 0x23) is obtained.
- The PCI Common Header is read from the device to get the first Base Address.

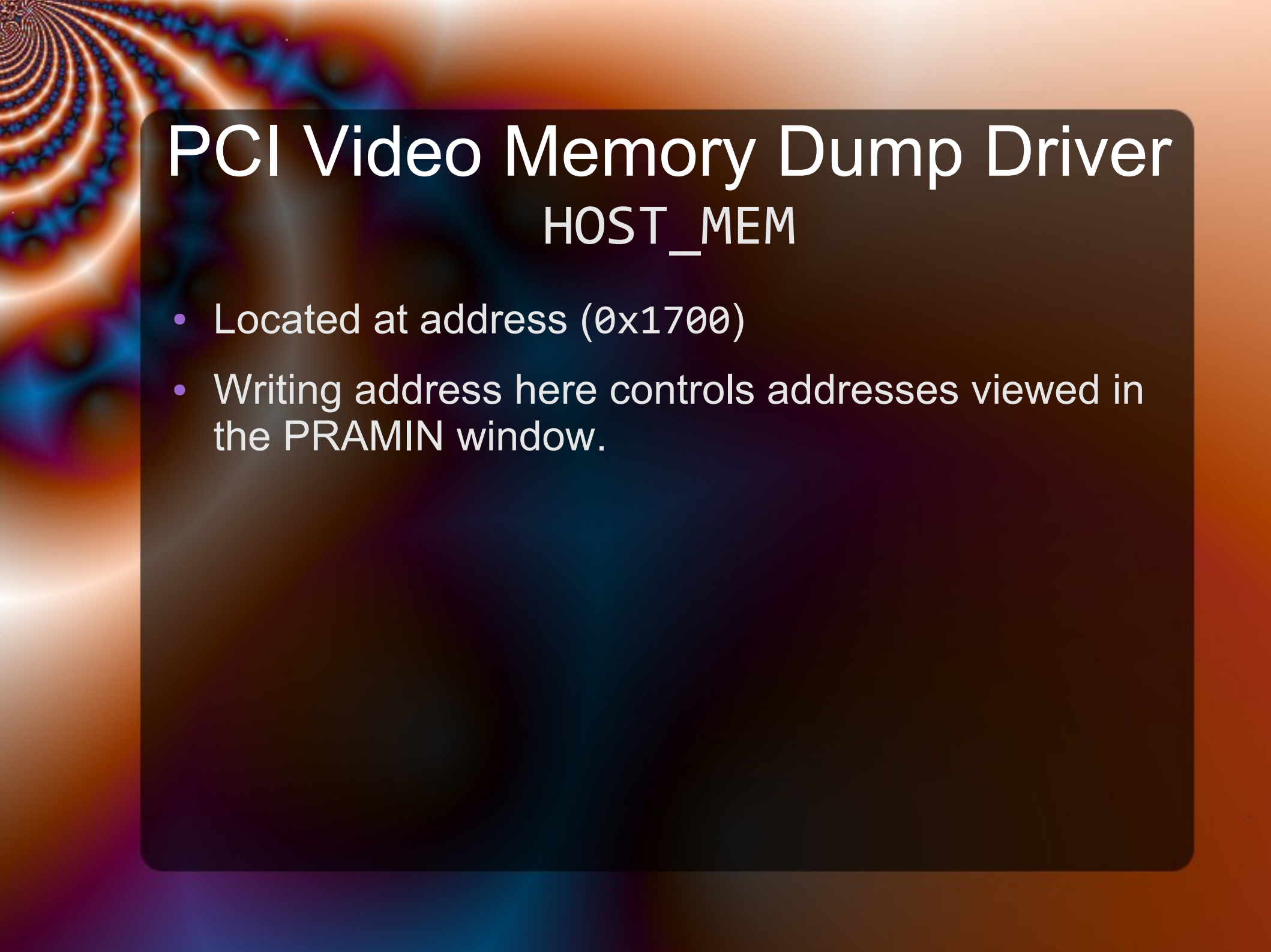
BAR 0

0x1700
(HOST_MEM)

0x700000
PRAMIN
0x7FFFFFFF Window

Video
Device
Memory





PCI Video Memory Dump Driver

HOST_MEM

- Located at address (0x1700)
- Writing address here controls addresses viewed in the PRAMIN window.



PCI Video Memory Dump Driver

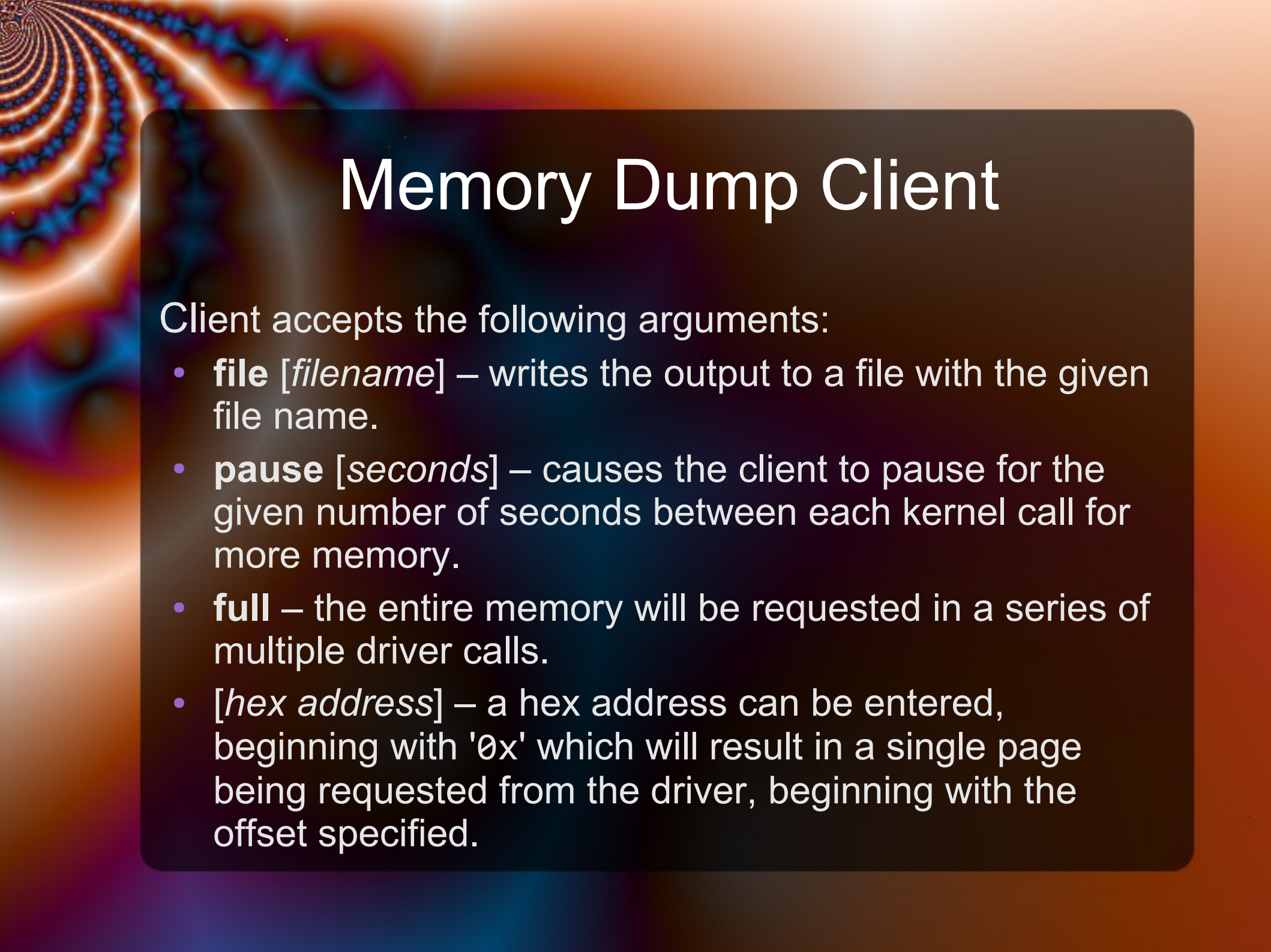
PRAMIN window

- Address range (0x700000 – 0x7FFFFFF)
- Read/Write access to the actual video memory.



Memory Dump Client

- The Dump Client calls the Control Device with offset values
 - Beginning at 0 and
 - Incrementing by 0x10
 - Continues it has pulled down the entire 1024 MB.



Memory Dump Client

Client accepts the following arguments:

- **file** [*filename*] – writes the output to a file with the given file name.
- **pause** [*seconds*] – causes the client to pause for the given number of seconds between each kernel call for more memory.
- **full** – the entire memory will be requested in a series of multiple driver calls.
- [*hex address*] – a hex address can be entered, beginning with '0x' which will result in a single page being requested from the driver, beginning with the offset specified.

Baseline Test

- No memory dump activities executing.
- Provide the goal for which any strategy needs to achieve to provide unobtrusive protection.

Ram Speed	5,575	MB/Second
CPU Tests	60,384,216	Floating Point Operations/Second
	150,253,162	Integer Operations/Second
	906,191	Hashes/Second
GPU Test	727	Frames/Second

No Pause Write

- Dump the entire GPU device memory to a file.
- Took an average of 5 minutes 51 seconds.
- Caused the entire system to slow down to the point where even moving the mouse was impacted.

Ram Speed	3,526	MB/Second
CPU Tests	4,967,279	Floating Point Operations/Second
	16,791,938	Integer Operations/Second
	67,803	Hashes/Second
GPU Test	64	Frames/Second

No Pause No Write

- Fetch memory from the video card but did not write to file.
- Required 5 minutes 44 seconds to run.
- Same performance impact as previous tests.

Ram Speed	2,137	MB/Second
CPU Tests	1,790,917	Floating Point Operations/Second
	5,915,372	Integer Operations/Second
	38,479	Hashes/Second
GPU Test	32	Frames/Second

10 Second Pause

- Adding a 10 second pause between each kernel call and writing the contents to disk.
- No appreciable degradation in system responsiveness when compared to the original baseline.
- Required 2 hours, 50 minutes, and 30 seconds to run.

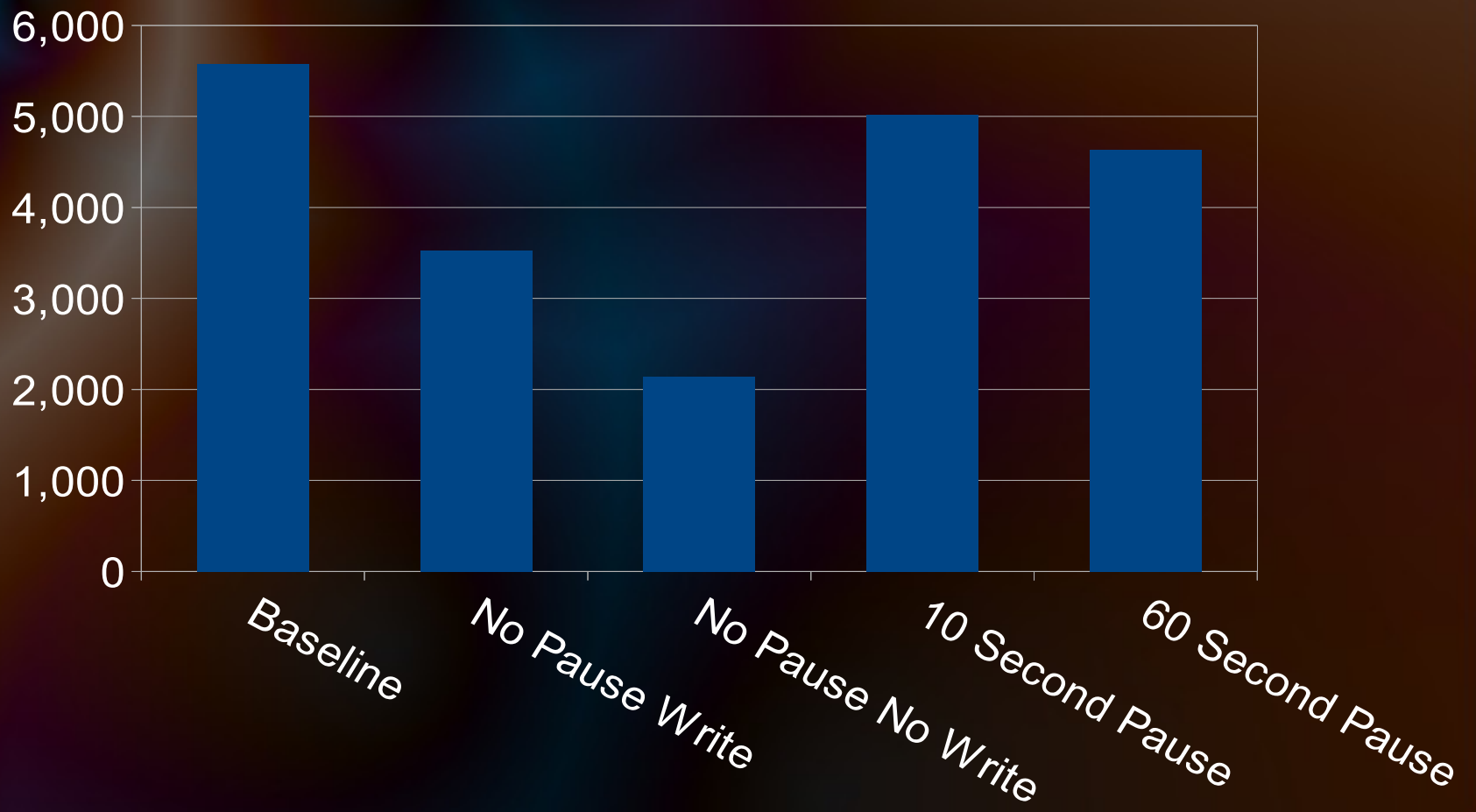
Ram Speed	5,022	MB/Second
CPU Tests	51,517,702	Floating Point Operations/Second
	145,357,310	Integer Operations/Second
	942,713	Hashes/Second
GPU Test	826	Frames/Second

60 Second Pause

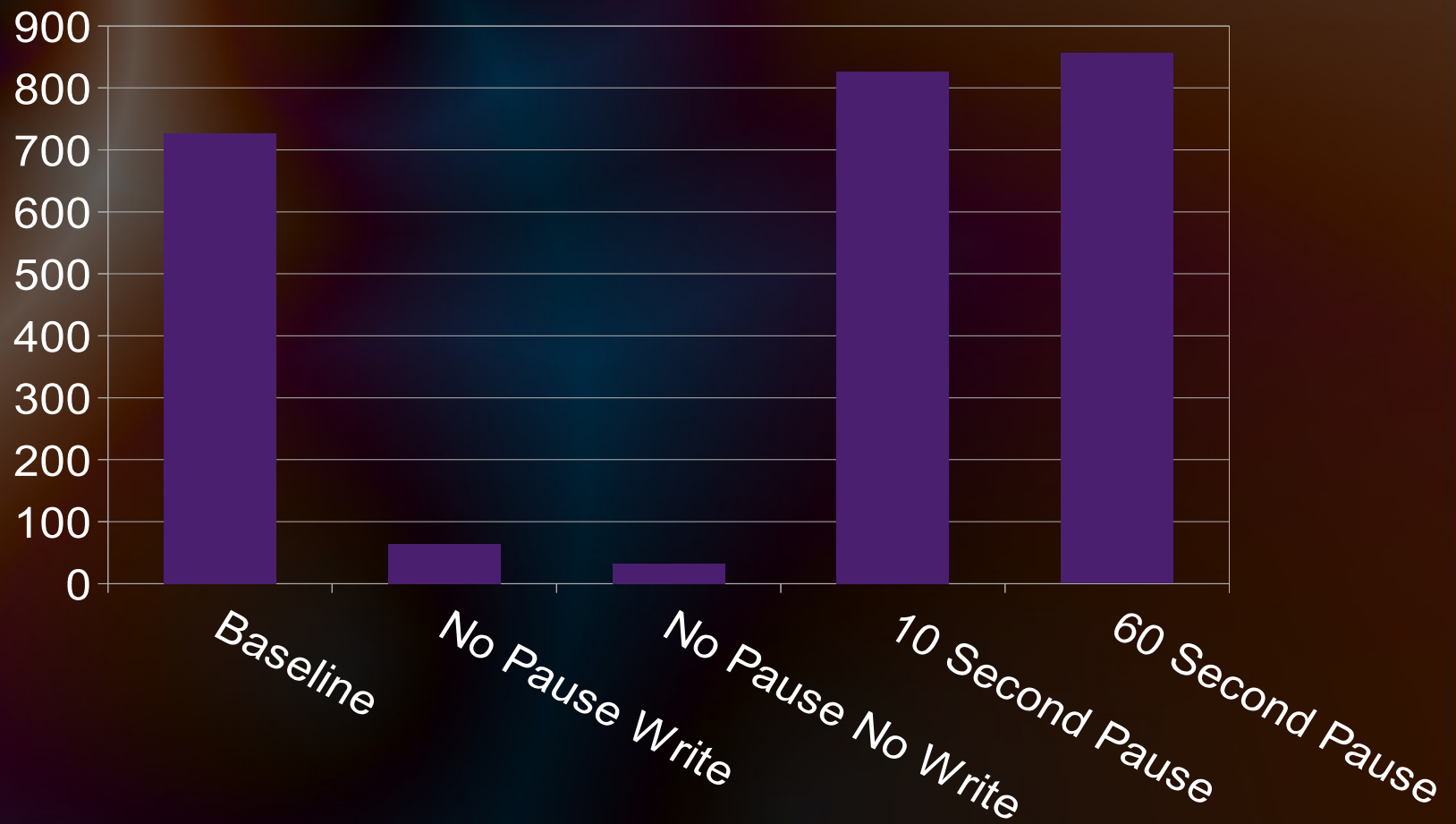
- Increased the pause between each kernel call to fetch memory to 60 seconds.
- Little appreciable difference in system responsiveness from previous test.
- Required almost 18 hours to complete.

Ram Speed	4,636	MB/Second
CPU Tests	60,765,344	Floating Point Operations/Second
	128,194,788	Integer Operations/Second
	943,973	Hashes/Second
GPU Test	856	Frames/Second

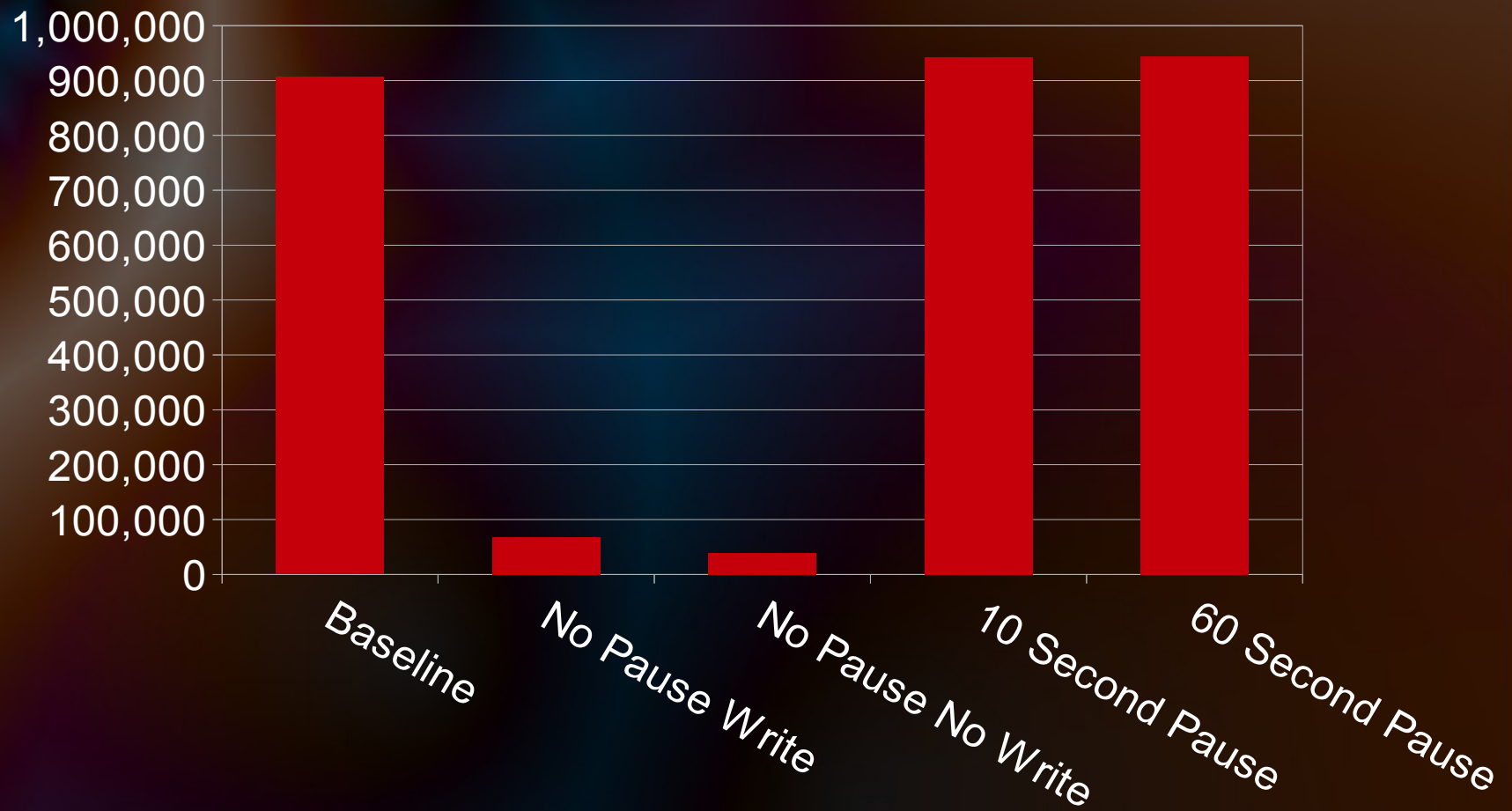
RAM MB/Second



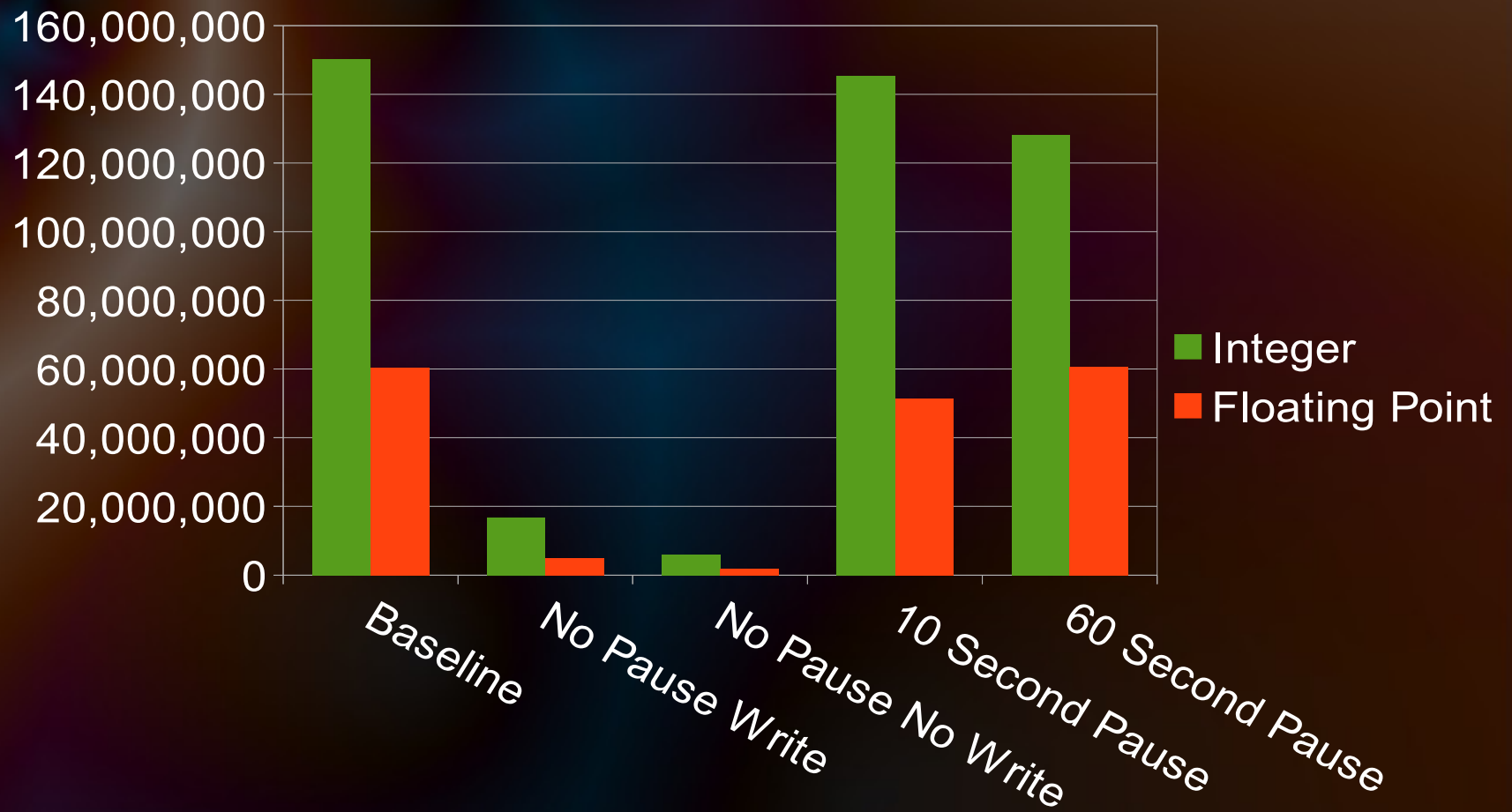
GPU Frames/Second

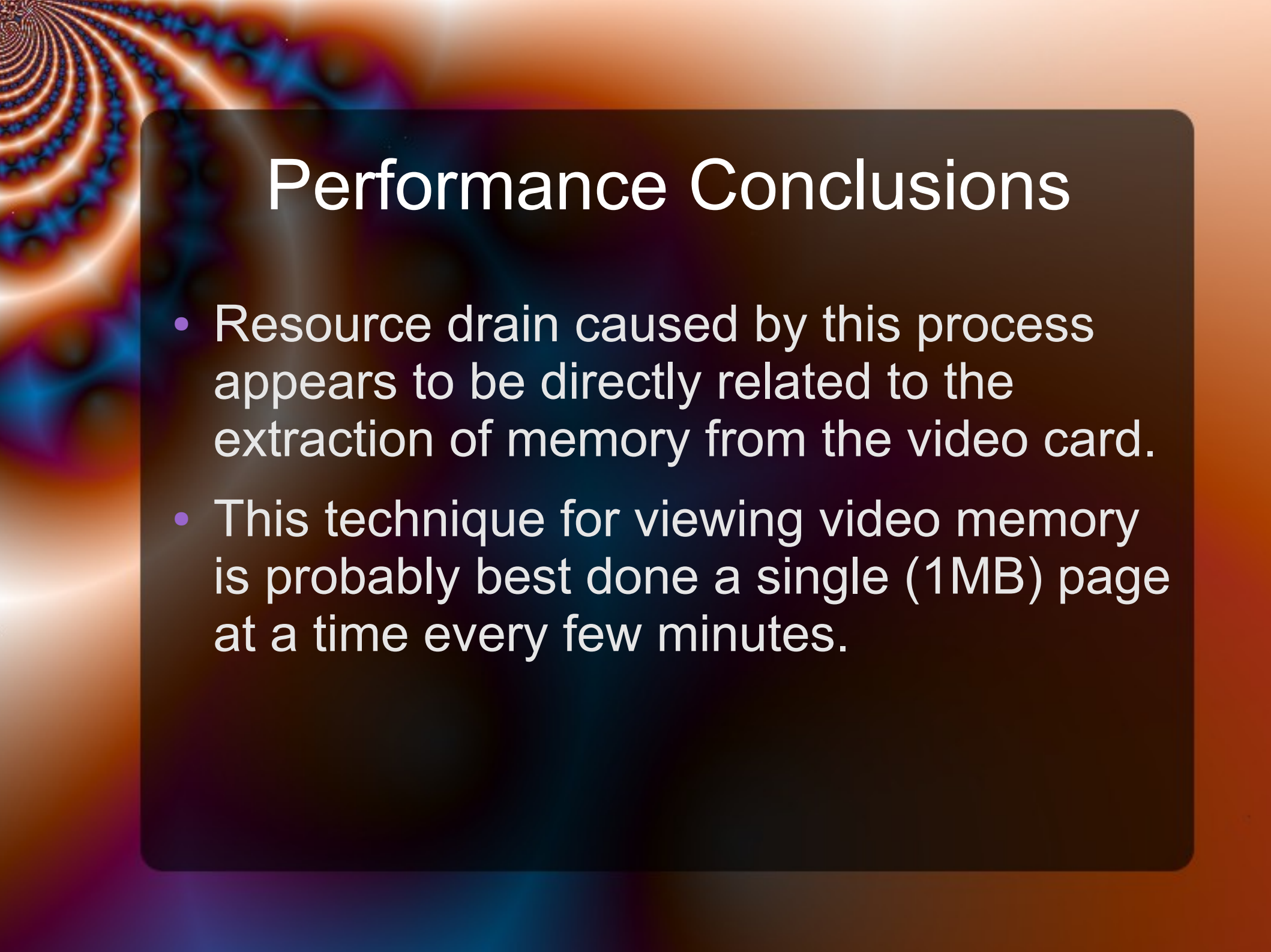


MD5Sum Hashes/Second



CPU Operations/Second



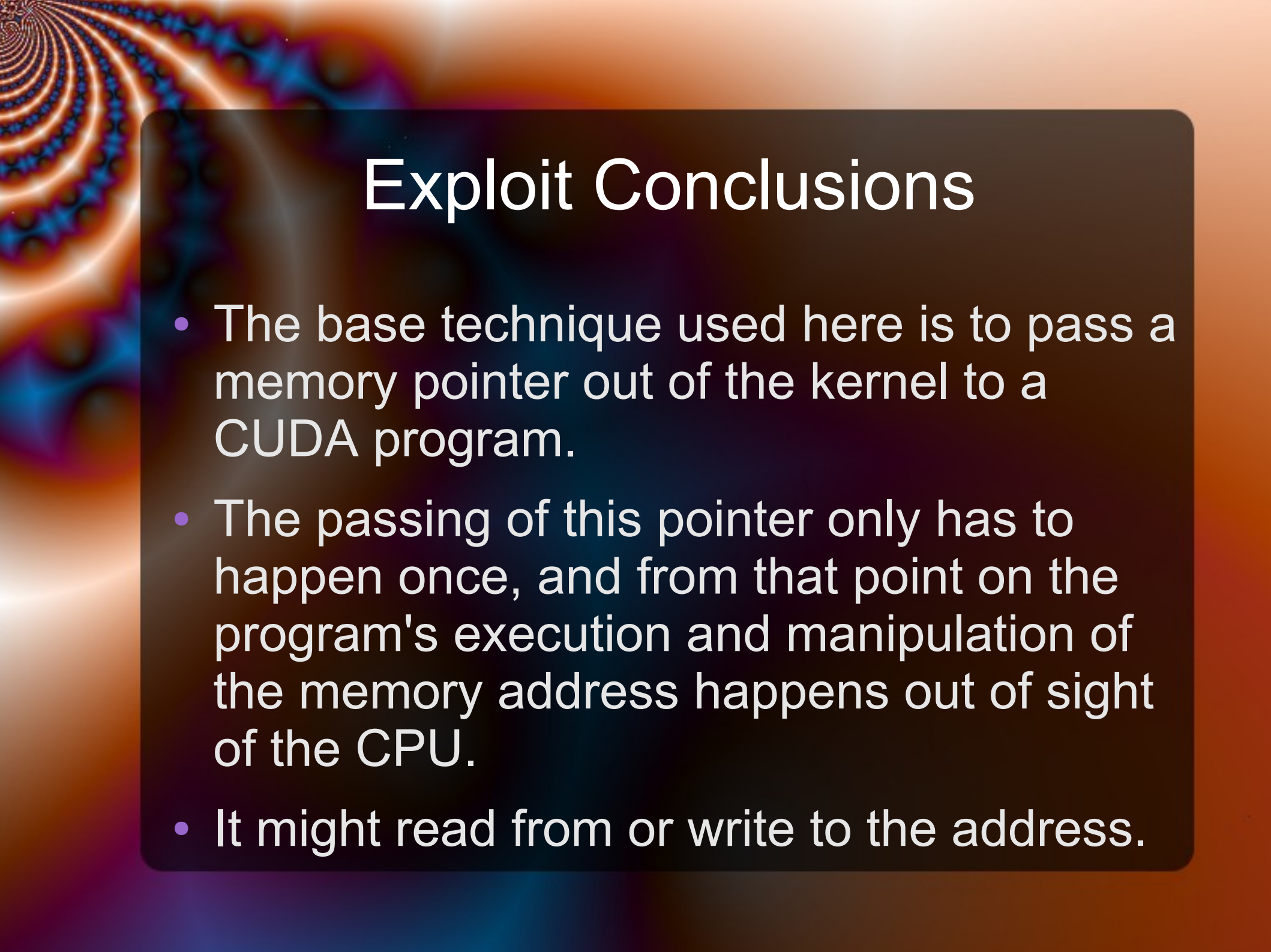


Performance Conclusions

- Resource drain caused by this process appears to be directly related to the extraction of memory from the video card.
- This technique for viewing video memory is probably best done a single (1MB) page at a time every few minutes.

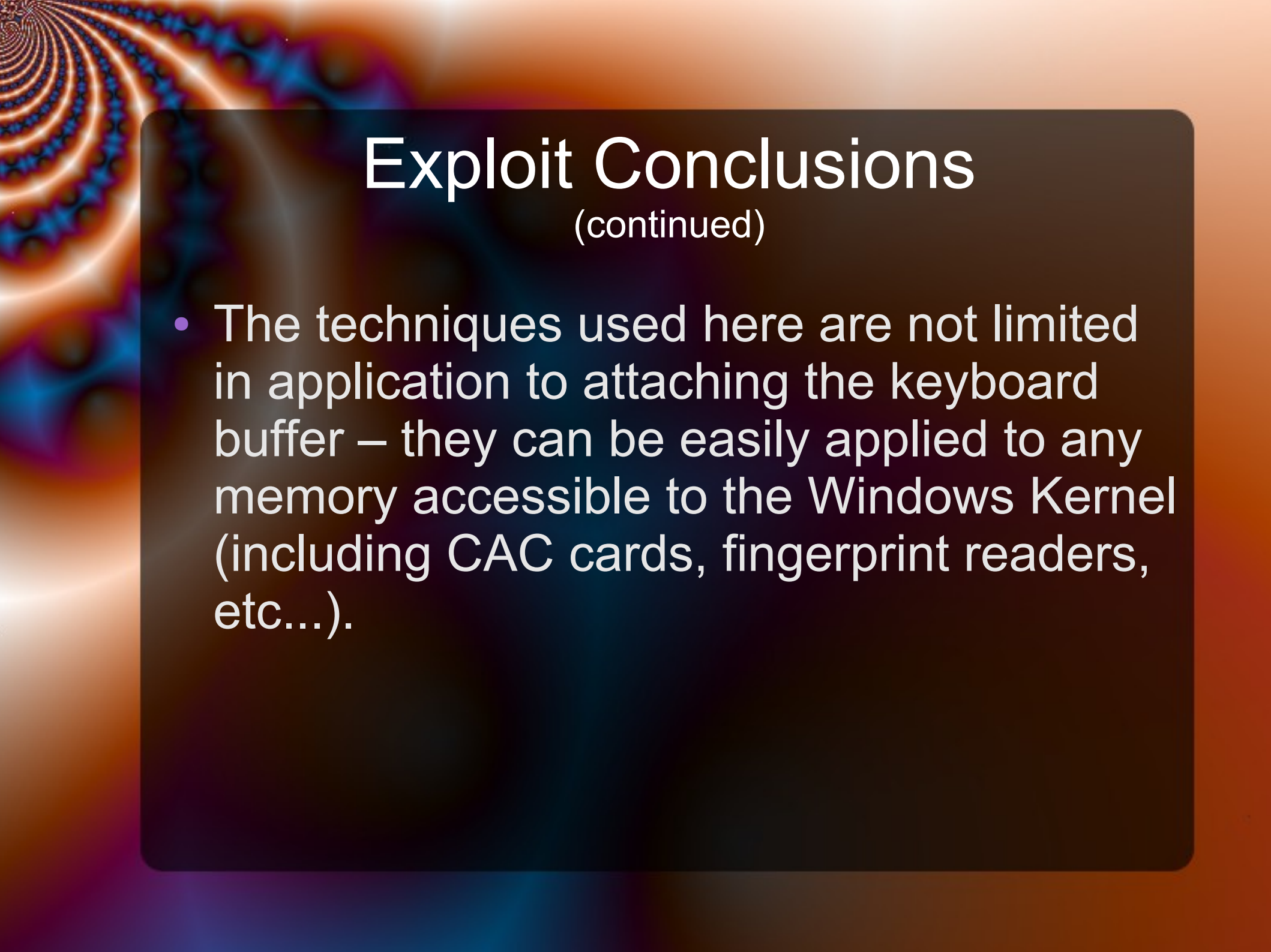
The background features a complex fractal pattern on the left side, with swirling orange and blue lines. A large, dark, rounded rectangle is centered on the page, serving as a backdrop for the title text.

Speculation And Future Work



Exploit Conclusions

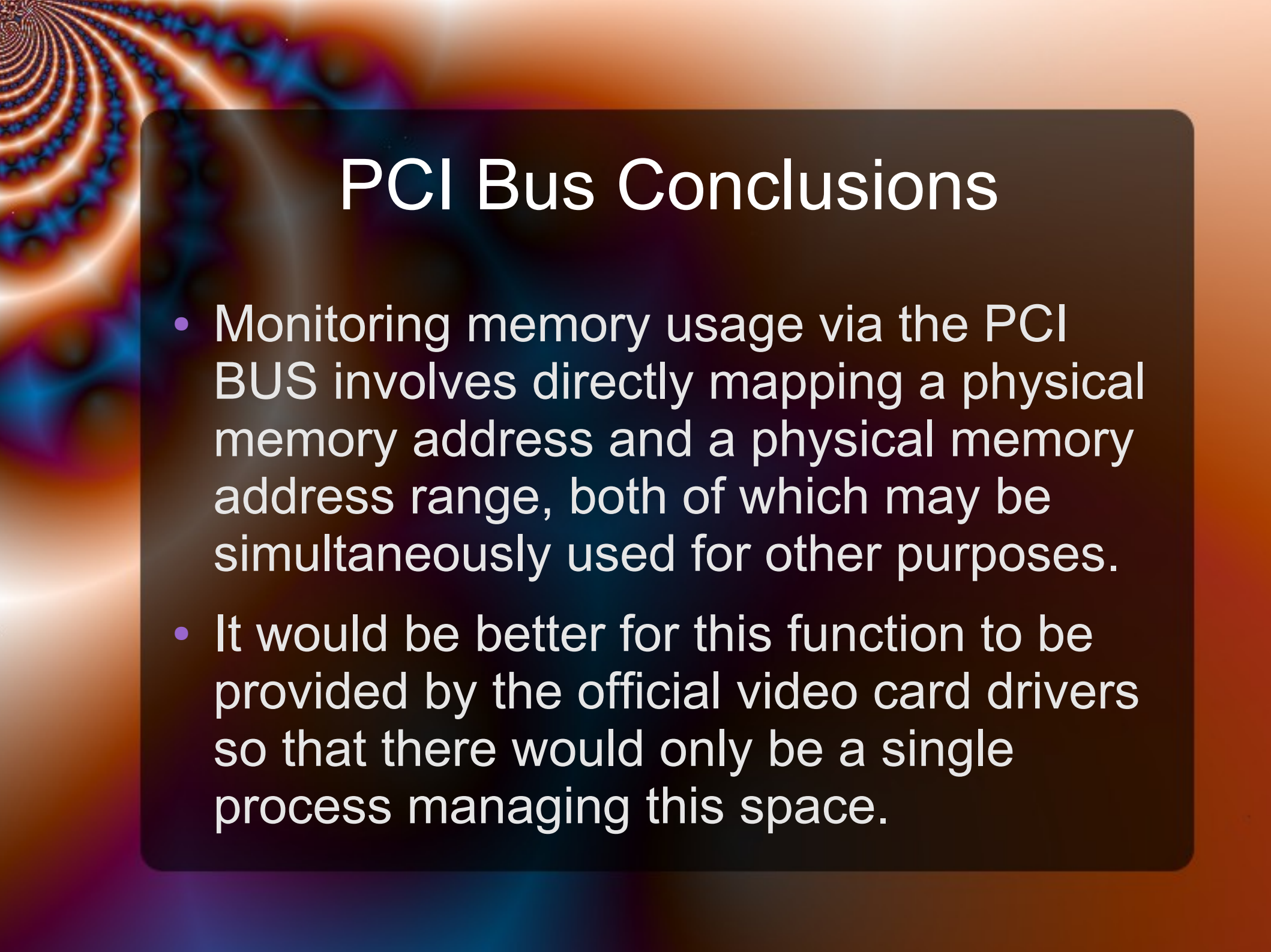
- The base technique used here is to pass a memory pointer out of the kernel to a CUDA program.
- The passing of this pointer only has to happen once, and from that point on the program's execution and manipulation of the memory address happens out of sight of the CPU.
- It might read from or write to the address.



Exploit Conclusions

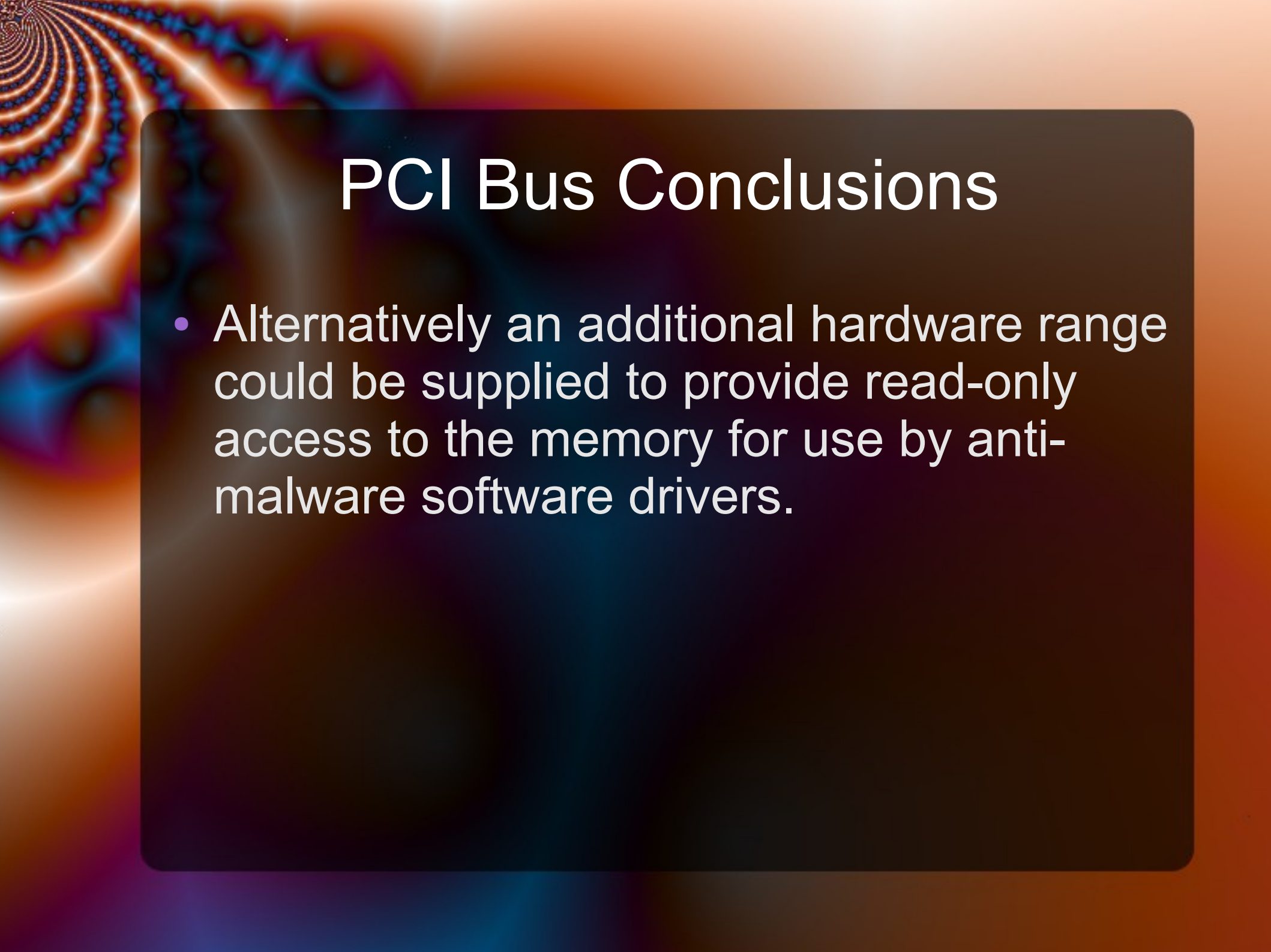
(continued)

- The techniques used here are not limited in application to attaching the keyboard buffer – they can be easily applied to any memory accessible to the Windows Kernel (including CAC cards, fingerprint readers, etc...).



PCI Bus Conclusions

- Monitoring memory usage via the PCI BUS involves directly mapping a physical memory address and a physical memory address range, both of which may be simultaneously used for other purposes.
- It would be better for this function to be provided by the official video card drivers so that there would only be a single process managing this space.



PCI Bus Conclusions

- Alternatively an additional hardware range could be supplied to provide read-only access to the memory for use by anti-malware software drivers.